

**Universidad Técnica Nacional**

**Sede San Carlos**



**Código Curso: ISW-1013**

**Nombre Curso: Calidad del Software**

**Análisis y Corrección de Vulnerabilidades**

**SQL Injection en Aplicación Web Real**

**Estudiantes:**

|                                 |                  |
|---------------------------------|------------------|
| <b>Gredy Corrales Mendoza</b>   | <b>208520626</b> |
| <b>Keiry Chacón Sibaja</b>      | <b>402670948</b> |
| <b>Deiber Molina Sobalbarro</b> | <b>208600495</b> |

**Profesor:**

**José David Carvajal Jiménez**

**Fecha: 23/3/2026**

## 1. Descripción del Sistema

La aplicación analizada es un sistema de gestión de libros desarrollado con Python/Flask y SQLite. Cuenta con cuatro módulos principales:

| Módulo    | Descripción                                                                                                      |
|-----------|------------------------------------------------------------------------------------------------------------------|
| Login     | Formulario de autenticación con usuario y contraseña.                                                            |
| Dashboard | Pantalla de bienvenida que muestra el usuario y rol activo.                                                      |
| Búsqueda  | Búsqueda de libros por título, autor o categoría.                                                                |
| Admin     | Panel de administración con lista de usuarios, contraseñas y log de auditoría. Solo accesible para el rol admin. |

## 2. Resumen de Vulnerabilidades Identificadas

Se identificaron 8 vulnerabilidades en la aplicación las cuales vemos en la siguiente tabla:

| ID   | Nombre                  | Categoría OWASP                 | Criticidad     | Archivo     |
|------|-------------------------|---------------------------------|----------------|-------------|
| V-01 | SQLi Login              | A03 – Injection                 | <b>CRÍTICA</b> | app.py      |
| V-02 | SQLi Búsqueda           | A03 – Injection                 | <b>CRÍTICA</b> | app.py      |
| V-03 | Contraseñas Texto Plano | A02 – Cryptographic Failures    | <b>ALTA</b>    | app.py      |
| V-04 | SECRET_KEY Hardcodeada  | A05 – Security Misconfiguration | <b>ALTA</b>    | app.py      |
| V-05 | Debug=True Activo       | A05 – Security Misconfiguration | <b>ALTA</b>    | app.py      |
| V-06 | Query SQL Expuesta      | A05 – Security Misconfiguration | <b>MEDIA</b>   | search.html |

| ID   | Nombre              | Categoría OWASP             | Criticidad   | Archivo           |
|------|---------------------|-----------------------------|--------------|-------------------|
| V-07 | Menú Admin Visible  | A01 – Broken Access Control | <b>MEDIA</b> | layout.html       |
| V-08 | Sin Protección CSRF | A01 – Broken Access Control | <b>MEDIA</b> | login/search.html |

### 3. Análisis Detallado por Vulnerabilidad

**V-01**

#### SQL Injection en el Login

#### Descripción del Problema

La ruta /login construye la consulta SQL concatenando directamente los valores que el usuario ingresa en los campos de usuario y contraseña. Esto permite a cualquier persona manipular la lógica de la consulta sin necesidad de conocer credenciales válidas.

#### Payload de Explotación

##### Ataque 1 – Bypass de contraseña (conociendo el usuario):

Campo Usuario: admin' --

Campo Contraseña: (cualquier texto)

La consulta resultante elimina la validación de contraseña:

```
SELECT id, username, role FROM users
```

```
WHERE username = 'admin' --' AND password = 'cualquiera'
```

##### Ataque 2 – Acceso sin conocer ningún usuario:

Campo Usuario: ' OR '1'='1' --

Campo Contraseña: (cualquier texto)

Esto hace que la condición WHERE sea siempre verdadera y retorna el primer registro de la tabla.

## Por qué es peligrosa

Un atacante puede ingresar al sistema con cualquier cuenta sin conocer la contraseña. Puede obtener acceso de administrador y comprometer completamente los datos del sistema. Es considerada una vulnerabilidad crítica por OWASP.

### Corrección:

| Código Vulnerable                                                                                                                                          | Código Corregido                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND password = '{password}'" user = conn.execute(query).fetchone()</pre> | <pre>user = conn.execute(     "SELECT id, username, password, role     FROM users WHERE username = ?",     (username,) ).fetchone()  if user and check_password_hash(     user["password"], password):</pre> |

**V-02**

## SQL Injection en Búsqueda

### Descripción del Problema

La ruta /search construye la consulta con LIKE insertando directamente el término de búsqueda. Un atacante puede salir del contexto de la consulta y ejecutar consultas mediante técnicas UNION-based.

### Payload de Explotación – Ataque UNION

```
%' UNION SELECT id, username, password, role FROM users --
```

Si la aplicación es vulnerable, en lugar de libros, los resultados muestran todos los usuarios de la base de datos con sus contraseñas en texto plano.

## Por qué es peligrosa

Permite filtrar cualquier tabla de la base de datos. Con contraseñas expuestas, el atacante puede realizar ataques de fuerza bruta offline o incluso acceder directamente si están en texto plano.

## Corrección

| Código Vulnerable                                                                                                                                         | Código Corregido                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>raw_query = f"SELECT id, title, author, category FROM books WHERE title LIKE '%{term}%' OR author LIKE '%{term}%' OR category LIKE '%{term}%'"</pre> | <pre>param = f"%{term}%" books = conn.execute( "SELECT id, title, author, category FROM books WHERE title LIKE ? OR author LIKE ? OR category LIKE ?", (param, param, param) ).fetchall()</pre> |

**V-03**

## Contraseñas en Texto Plano

## Descripción del Problema

Las contraseñas se almacenan en la tabla users como texto plano sin ningún tipo de transformación criptográfica. Cualquier acceso a la base de datos expone todas las credenciales inmediatamente.

## Evidencia de Explotación

Ingresando como admin al Panel de Administración, la columna password muestra directamente:

## Por qué es peligrosa

Si un atacante obtiene la base de datos, tiene acceso inmediato a todas las contraseñas sin necesidad de crackeo. Además, muchos usuarios reutilizan contraseñas en otros servicios, amplificando el daño.

## Corrección

Se utiliza `werkzeug.security` para generar hashes PBKDF2-SHA256 al crear usuarios y `check_password_hash` para validar en el login:

| Código Vulnerable                                                                                                                                       | Código Corregido                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre># Al insertar usuarios: users = [     ("admin", "Admin123", "admin"),     ... ]  # Al validar login: WHERE username='{u}' AND password='{p}'</pre> | <pre>from werkzeug.security import (     generate_password_hash,     check_password_hash)  # Al insertar: generate_password_hash("Admin123")  # Al validar: check_password_hash(     user["password"], password)</pre> |

**V-04**

**SECRET\_KEY Hardcodeada en el Código Fuente**

## Descripción del Problema

La clave secreta de Flask está escrita directamente en el código fuente: `app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"`. Si el repositorio es público o alguien accede al código, puede firmar cookies de sesión falsas y suplantar cualquier usuario.

## Por qué es peligrosa

Flask usa la `SECRET_KEY` para firmar cookies de sesión con HMAC. Conociendo esta clave, un atacante puede forjar una cookie válida con `role=admin` sin necesidad de autenticarse.

## Corrección Aplicada

| Código Vulnerable                                                            | Código Corregido                                                                                                                                                              |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>app.config["SECRET_KEY"] =<br/>    "dev-secret-key-insegura-1234"</pre> | <pre>import os<br/>app.config["SECRET_KEY"] =<br/>    os.environ.get(<br/>        "SECRET_KEY",<br/>        "fallback-solo-dev"<br/>    )<br/>- SECRET_KEY=clave-segura</pre> |

**V-05**

**Modo debug=True Activo en Producción**

## Descripción del Problema

Flask tiene debug=True activo en el arranque de la aplicación. En modo debug, cuando ocurre un error en el servidor, Flask muestra un debugger interactivo en el navegador que permite ejecutar código Python arbitrario en el servidor.

## Evidencia

## Por qué es peligrosa

Cualquier error no controlado (como una consulta SQL malformada) puede exponer el debugger. Desde ahí un atacante puede ejecutar comandos del sistema operativo, leer archivos y tomar control total del servidor.

## Corrección Aplicada

| Código Vulnerable                                                                       | Código Corregido                                                                             |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <pre>if __name__ == "__main__":<br/>    init_db()<br/>    app.run(host="0.0.0.0",</pre> | <pre>if __name__ == "__main__":<br/>    init_db()<br/>    debug_mode = os.environ.get(</pre> |

|                                   |                                                                                                           |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <pre>port=5000, debug=True)</pre> | <pre>"FLASK_DEBUG","false" ).lower() == "true" app.run(host="0.0.0.0", port=5000, debug=debug_mode)</pre> |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------|

## V-06

## Consulta SQL Expuesta en la Interfaz

### Descripción del Problema

El template search.html recibe la variable raw\_query y la muestra directamente en pantalla después de cada búsqueda. Esto revela la estructura exacta de las tablas de la base de datos y facilita enormemente la construcción de ataques.

### Evidencia

Al hacer cualquier búsqueda, el usuario ve en pantalla:

```
SELECT id, title, author, category FROM books
WHERE title LIKE '%flask%' OR author LIKE '%flask%'
OR category LIKE '%flask%'
```

### Corrección Aplicada

| app.py Vulnerable                                                                                      | app.py Corregido                                                                                |
|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <pre>return render_template(     "search.html",     books=books,     raw_query=raw_query # ← MAL</pre> | <pre>return render_template(     "search.html",     books=books     # raw_query eliminado</pre> |

Además, se eliminó el bloque HTML en search.html que renderizaba la variable.



**V-07****Enlace Admin Visible para Todos los Roles****Descripción del Problema**

El archivo layout.html muestra el enlace al panel Admin en la barra de navegación para todos los usuarios autenticados, sin importar su rol. Aunque el backend bloquea el acceso, la interfaz no debe exponer opciones que el usuario no tiene derecho a usar.

**Por qué es peligrosa**

Revela la existencia de un panel administrativo a usuarios no autorizados, incentivando intentos de acceso o escalada de privilegios. Viola el principio de mínima exposición de información.

**Corrección Aplicada**

| layout.html Vulnerable                                                                                                                                        | layout.html Corregido                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;a href="/dashboard"&gt;Dashboard&lt;/a&gt; &lt;a href="/search"&gt;Búsqueda&lt;/a&gt; &lt;a href="/admin"&gt;Admin&lt;/a&gt; ← siempre visible</pre> | <pre>&lt;a href="/dashboard"&gt;Dashboard&lt;/a&gt; &lt;a href="/search"&gt;Búsqueda&lt;/a&gt; {% if session.get('role')     == 'admin' %}     &lt;a href="/admin"&gt;Admin&lt;/a&gt; {% endif %}</pre> |

**V-08****Ausencia de Protección CSRF****Descripción del Problema**

Los formularios de login y búsqueda no incluyen tokens CSRF. Esto permite que un sitio externo malicioso envíe solicitudes en nombre de un usuario autenticado sin su consentimiento (Cross-Site Request Forgery).

## Escenario de Explotación

Un usuario autenticado visita un sitio malicioso. Ese sitio tiene un formulario oculto que apunta a la aplicación. Al cargar la página, el formulario se envía automáticamente usando las cookies activas de la víctima.

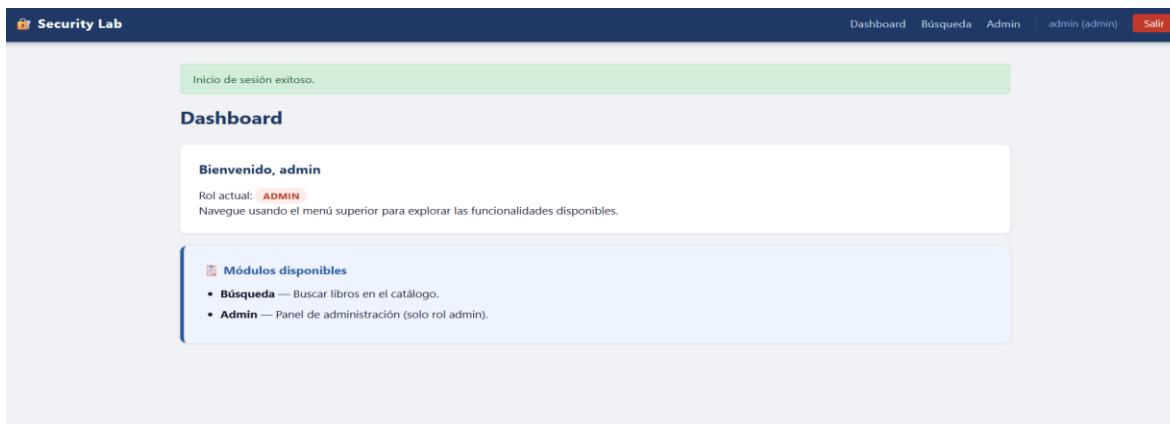
## Corrección Aplicada

Se instaló Flask-WTF y se habilitó CSRFProtect globalmente. Cada formulario incluye ahora un token único:

| Formulario Vulnerable                                                                                                                                                                       | Formulario Corregido                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;form method="post"&gt;   &lt;!-- sin token CSRF --&gt;   &lt;input name="username"&gt;   &lt;input name="password"&gt;   &lt;button&gt;Ingresar&lt;/button&gt; &lt;/form&gt;</pre> | <pre>from flask_wtf.csrf import CSRFProtect csrf = CSRFProtect(app)  # En el HTML: &lt;form method="post"&gt;   &lt;input type="hidden"     name="csrf_token"     value="{{ csrf_token() }}"&gt;   ... &lt;/form&gt;</pre> |

## 4. Verificación de las Correcciones

### Ataque 1: Bypass de login sin contraseña (V-01)



### Ataque 2: Login sin conocer ningún usuario (V-01)



### Ataque 3: Extraer usuarios con UNION (V-02)

Resultados (11 encontrados)

| ID | Título                       | Autor       | Categoría   |
|----|------------------------------|-------------|-------------|
| 1  | Secure Coding Fundamentals   | J. Howard   | security    |
| 1  | admin                        | Admin123    | admin       |
| 2  | Flask Web Patterns           | A. Miller   | development |
| 2  | analyst                      | Analyst123  | user        |
| 3  | Practical SQL                | A. DeBarros | database    |
| 3  | student                      | Student123  | user        |
| 4  | Threat Modeling Essentials   | M. Silva    | security    |
| 5  | Performance Testing Handbook | R. Jones    | qa          |
| 6  | OWASP Testing Guide          | OWASP Team  | security    |
| 7  | Clean Code                   | R. Martin   | development |
| 8  | The Web Application Hacker   | Stuttard    | security    |

#### Búsqueda de Libros

Término de búsqueda

%' UNION SELECT id, username, password, role FROM users --

Buscar

#### Búsqueda de Libros

Término de búsqueda

Buscar por título, autor o categoría...

Buscar

### Ataque 4: Verificar consulta expuesta (V-06)

#### Búsqueda de Libros

Término de búsqueda

Buscar por título, autor o categoría...

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

```
SELECT id, title, author, category FROM books WHERE title LIKE '%SQL%' OR author LIKE '%SQL%' OR category LIKE '%SQL%'
```

Resultados (1 encontrados)

| ID | Título        | Autor       | Categoría |
|----|---------------|-------------|-----------|
| 3  | Practical SQL | A. DeBarros | database  |

 **Búsqueda de Libros**

Término de búsqueda

Buscar por título, autor o categoría...

Buscar

Resultados (1 encontrados)

| ID | Título        | Autor       | Categoría |
|----|---------------|-------------|-----------|
| 3  | Practical SQL | A. DeBarros | database  |

## Ataque 5: Contraseñas en texto plano (V-03)

 **Panel de Administración**

Usuarios del sistema

⚠ Las contraseñas se muestran en texto plano — esto es una vulnerabilidad intencional (V-03).

| ID | Usuario | Contraseña (texto plano ⚠) | Rol   |
|----|---------|----------------------------|-------|
| 1  | admin   | Admin123                   | ADMIN |
| 2  | analyst | Analyst123                 | USER  |
| 3  | student | Student123                 | USER  |

 **Panel de Administración**

Usuarios del sistema

⚠ Las contraseñas se muestran en texto plano — esto es una vulnerabilidad intencional (V-03).

| ID | Usuario | Contraseña (texto plano ⚠)                                                                                                                                           | Rol   |
|----|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1  | admin   | scrypt:32768:8:1\$ruR7nRgU0i0iDn03\$5092a793c4f98dc92925d7b034848d714b5a5a0c89b28b086f1a68c768dbbf517cfe23b5452ad24ffc7572daee813f664dfc7d2c83a9503abe9dacefd9d5292  | ADMIN |
| 2  | analyst | scrypt:32768:8:1\$qtFk98Q0HBjaG0qa\$1f1311512e1134ec5ef6ba61c02fdb132431660e032900223d27e765b8f0b153ce450d6c29b9d509a01c6d3056177c945cb5d4c4f6c65fe4b7f58755649bc0ed | USER  |
| 3  | student | scrypt:32768:8:1\$xfmI2vrrr808Nag\$bf42dc7ce63ceef43140fb5156105a263b9230c21946483ce2cdba04eaf880c1abc092e29642313cbaaea5926beddf9a006941aca6fe789f2c7ad9df669edb3   | USER  |

## 5. Conclusiones

### ¿Qué aprendimos?

Este laboratorio nos permitió comprender de manera práctica por qué las vulnerabilidades de inyección SQL son consideradas históricamente las más peligrosas en aplicaciones web. Ver la aplicación vulnerable en funcionamiento y ejecutar los payloads de explotación fue mucho más ilustrativo que solo leer una presentación o ver la teoría.

Aprendimos que la causa raíz de V-01 y V-02 es exactamente la misma: confiar en lo que el usuario inserta y que puede mezclarlo con instrucciones SQL. Las soluciones son simples, pero requieren un cambio de mentalidad, el dato del usuario nunca debe ser parte del texto de la consulta.

También entendimos la relación entre vulnerabilidades: V-02 (SQLi en búsqueda) combinada con V-03 (contraseñas en texto plano) crea un impacto crítico, ya que el atacante no solo puede extraer datos sino obtener credenciales directamente usables.

### **¿Qué fue lo más difícil?**

La corrección de V-03 fue la más compleja porque requiere cambios en dos lugares distintos del código (al crear usuarios y al validar el login) y además implica que la base de datos existente debe regenerarse. Si se cambia solo la validación, pero no la creación, los usuarios existentes quedan con contraseñas en texto plano y el `check_password_hash` falla.

V-08 (CSRF) también presentó dificultades porque requiere instalar una dependencia nueva (Flask-WTF), configurarla correctamente con CSRFProtect, y modificar cada formulario HTML. Un error en cualquiera de esos pasos rompe el formulario completamente.

### **¿Cuál vulnerabilidad es más crítica?**

Consideramos que V-01 (SQLi en Login) es la vulnerabilidad más crítica porque permite a un atacante sin ningún conocimiento previo tomar control total de la aplicación en segundos. No requiere herramientas especiales, solo un navegador y el payload correcto. El impacto es inmediato, un acceso de administrador, que luego abre la puerta a explotar todas las demás vulnerabilidades desde adentro.

En segundo lugar, V-03 (contraseñas en texto plano) es crítica porque es un daño permanente, ya que una vez que alguien obtiene una copia de la base de datos, las contraseñas quedan expuestas para siempre, independientemente de que se corrijan las demás fallas.

### **LINK REPOSITORIO:**

[https://github.com/Gredy-Co/sqli\\_utn/tree/feature/gredycorrales\\_keirychacon\\_deibermolina](https://github.com/Gredy-Co/sqli_utn/tree/feature/gredycorrales_keirychacon_deibermolina)